

F1 Team Radio NLP Project - Viva Q&A Guide

Hochschule Aalen - M.Sc. Machine Learning & Data Analytics

Student: Shashwat Singh

1. Who built this project and what is its academic context?

This project was designed and implemented by Shashwat Singh, a student of the M.Sc. Machine Learning and Data Analytics program at Hochschule Aalen. The project satisfies curriculum requirements in Natural Language Processing (NLP), demonstrating end-to-end data acquisition, statistical text analysis, classical machine learning classification, advanced sequence models (RNN/LSTM), custom named entity recognition, and RAG pipelines.

2. How was the data sourced and what is its volume?

The dataset was collected using a custom-built BeautifulSoup web scraper that targeted the AutoLAP team radio transcripts archive (transcripts.recurseprojects.cloud). It crawled three full Grand Prix sessions from 2026: Barcelona, Monaco, and Montreal. The total corpus size is 18,574 individual message records, which fits precisely in the M.Sc. project guideline requirement of 1,000 to 30,000 rows.

3. What is the difference between the Interactive Chatbot and the RAG Simulator?

For the Interactive Chatbot: Operates on rule-based patterns and statistical transition probabilities. It matches query keywords to return canned F1 answers and utilizes a Trigram Language Model to generate synthetic text.

For the RAG Simulator (Retrieval-Augmented Generation): An advanced pipeline that searches 5 distinct databases (Radios, Reports, Regulations, Teams, specs) in parallel. It retrieves the top-k relevant rows using keyword search, aggregates them into a structured context window, and dynamically formats an LLM prompt template for a race strategist persona.

4. How does the TF-IDF search engine work?

The search engine is built from scratch. For a user query:

1. It calculates Term Frequency (TF) representing word frequency in a document/query.
2. It scales this by Inverse Document Frequency (IDF), indicating word uniqueness across all 18,574 messages.
3. It computes the manual Cosine Similarity between the query vector and document vectors:

$$\text{Similarity} = (Q \cdot D) / (\|Q\| * \|D\|)$$

4. Finally, it sorts and returns the top-k highest similarity score documents.
-

5. What are Zipf's Law and Luhn's Cut in your analysis?

Zipf's Law states that word frequency is inversely proportional to its rank. On a log-log plot of our F1 vocabulary, we verified a clear linear distribution, proving Zipfian behavior.

Luhn's Cut specifies that high-frequency words (stopwords) and low-frequency words (typos, unique names) have low indexing value. We filter them out to focus on mid-frequency, high-information terms (e.g. 'box', 'lap', 'degradation').

6. Why did you use Sparse Matrices (CSR/CSC)?

A dense matrix containing 18,574 rows and a vocabulary of several thousand unique words would consume massive memory (~500MB+ in memory) and compute slowly. By converting representations to SciPy CSR (Compressed Sparse

F1 Team Radio NLP Project - Viva Q&A Guide

Hochschule Aalen - M.Sc. Machine Learning & Data Analytics

Student: Shashwat Singh

Row) and CSC (Compressed Sparse Column) formats, we store only non-zero occurrences, achieving a 30x memory reduction and significantly faster matrix multiplication speeds.

7. How did you train the custom spaCy Named Entity Recognition (NER) model?

Standard English models do not recognize F1 jargon. We trained a custom model by:

1. Initializing a blank English model with `spacy.blank('en')` and adding a fresh 'ner' pipeline component.
 2. Registering custom labels: DRIVER, TEAM, TIRE_COMPOUND, and STRATEGY.
 3. Creating annotated training samples with exact character start-end offsets.
 4. Running a 15-epoch training loop using Adam optimizer, shuffling the dataset at each epoch, and applying a 35% dropout rate to prevent overfitting.
-

8. How does the custom Sentiment Lexicon work?

Standard sentiment analyzers (VADER, TextBlob) fail on radio text, classifying 'box' (pit stop) or 'copy' (received) as generic terms. We created a custom F1 lexicon dictionary assigning positive weights to celebratory words ('p1', 'great', 'won') and negative weights to stress words ('wtf', 'slow', 'problem'). The user can dynamically adjust these weights in the Streamlit sidebar tuner, which updates the corpus sentiment scores and plots the driver timelines in real time.

9. What Deep Learning models did you build in PyTorch?

CBOW (Continuous Bag of Words): Learns 50-dimensional F1 word embeddings from scratch, mapping words to semantic clusters (e.g. 'box' clusters near 'pit').

PyTorch MLP (Multi-Layer Perceptron): A feed-forward network trained on TF-IDF features to categorize messages into Strategy, Position/Gap, Emotional, or Technical.

PyTorch RNN and LSTM: Recurrent neural networks trained on padded token-index sequences to capture sequential dependencies in radio messages.

10. What is the performance hierarchy of your text classifiers?

Classical and Deep Learning classifiers were trained and benchmarked:

Linear SVM: 93.81% accuracy (highest overall due to effective hyper-plane separation of sparse TF-IDF vectors).

PyTorch MLP: 92.57% accuracy.

Logistic Regression: 92.21% accuracy.

PyTorch LSTM: 81.42% accuracy (limited by short telegraphic lengths of radio messages).

PyTorch RNN: 79.99% accuracy.

Naive Bayes: 79.94% accuracy.
